

Nova Kata Platform: Detailed Architecture & Traffic Flow Analysis

1. System Overview

The Nova Kata platform is a containerised serverless infrastructure designed to execute user functions with the isolation of micro-VMs but the speed of standard containers. It accomplishes this by utilizing **Kata Containers** (running via QEMU) and a **"Warm Pool" strategy** that effectively eliminates the classic serverless "cold start" problem.

The platform is divided into three primary components:

1. **nova-dashboard**: The frontend user interface (Next.js).
2. **nova-kata (Control Plane / Placement Service)**: The core orchestrator that manages Worker VMs and container lifecycles.
3. **nova-kata-gateway (Data Plane)**: The API gateway that securely routes traffic to the correct executing functions.

This document focuses on the backend (`nova-kata`), the proxy/routing (`nova-kata-gateway`), and the underlying deployment and execution mechanics.

2. Backend Architecture Details

2.1. `nova-kata` (The Control Plane)

`nova-kata` is a Node.js Express service (default port `3002`) that serves as the brain of the cluster. It does not execute functions directly; instead, it manages the infrastructure that does.

Key Responsibilities:

- **Worker VM Management (`workerService.js` & `gcpService.js`)**: It manages a fleet of Worker VMs. It supports auto-scaling by hooking into Google Cloud Platform (GCP) to provision (`n2-standard-2`) instances that support nested virtualization.
- **Container Lifecycle (`containerService.js`)**: It communicates with Worker VMs strictly over SSH. It generates shell scripts on the fly to execute `nerdctl` commands on the workers to launch, pause, unpause, and stop Kata containers.
- **Warm Pool Management (`warmPoolService.js`)**: To avoid cold starts, `nova-kata` maintains a background pool of containers. When a container is launched, it is immediately paused using `nerdctl pause` . The pool size is governed by `.env` parameters (`WARM_POOL_MIN` , `WARM_POOL_MAX`).
- **State Persistence**: It uses a local SQLite database (`nova-kata.db`) to track the status of workers, active containers, the warm pool inventory, and deployed functions.

2.2. `nova-kata-gateway` (The Data Plane)

`nova-kata-gateway` is a dual-port Node.js Express service running on port `8081` (Public facing) and `3003` (Internal control plane).

Key Responsibilities:

- **Request Interception**: It receives HTTP requests destined for serverless functions.

- **Resolution & Authentication (`parseHost.js` , `authCheck.js`):** It identifies the requested function via the `Host` header (or path mapping) and verifies API keys.
 - **Container Claiming (`containerStateCheck.js`):** Before proxying data, the gateway makes an internal API call to `nova-kata` (`POST /execute`). `nova-kata` immediately "unpauses" a container from the warm pool and returns the target IP and Port.
 - **Dynamic Proxying (`forwardRequest.js`):** The gateway uses `http-proxy-middleware` to forward the HTTP request to the specific worker node and port assigned to the unpaused container.
-

3. Deployment & Infrastructure Pipeline

The platform does not rely on Kubernetes or standard Docker daemon. It utilizes a highly specialized container stack designed for multi-tenant security.

3.1. The Container Stack on Worker VMs

- **Engine:** `containerd` is the core daemon running on workers.
- **CLI:** `nerdctl` is used instead of Docker. It is a Docker-compatible CLI specifically tailored for `containerd`.
- **Runtime:** `io.containerd.kata.v2` . This is the crucial security layer. Instead of running a container as a standard Linux process, Kata wraps the container inside a lightweight QEMU virtual machine.
- **Snapshotter:** `overlayfs` .

3.2. Networking Setup per Container

When `nova-kata` launches a container on a Worker VM, it performs a highly specific networking dance:

1. **Launch Container:** It runs `nerdctl run ...` on the worker. The container gets an internal CNI IP address (e.g., `10.4.0.5`).
2. **Port Allocation:** `nova-kata` allocates an unused port on the Worker VM's host OS (e.g., `9005`).
3. **Nginx Bridge:** `nova-kata` dynamically writes an `nginx` configuration block directly on the worker to reverse-proxy traffic arriving at host port `9005` into the container's internal IP at its designated application port (usually `8080`).

3.3. GCP Auto-Scaling

`nova-kata` runs a background cron job (`schedulerService.js` / `scalingService.js`) that monitors the overall utilization of the cluster (how many containers are active vs. how many workers exist). If utilization exceeds `SCALE_OUT_THRESHOLD` (e.g., 75%), it automatically calls the GCP API to spin up a new Worker VM, injects a startup script to install the required stack (`containerd`, `Kata`, `Nginx`), and registers it into the pool.

4. End-to-End Traffic Flow (The "No Cold Start" Lifecycle)

The magic of the platform is how quickly it serves requests despite using Virtual Machines.

1. **Background Replenishment:** Long before a user makes a request, `nova-kata` has SSH'd into workers, run `nerdctl run` , let the Kata VM boot (which takes 5-10 seconds), and immediately run `nerdctl pause` . The VM is frozen in memory.

2. **Inbound Request:** A user sends an HTTP request to `function-abc.nova.local` targeting the gateway (`8081`).
3. **Gateway Intercept:** The gateway pauses the request and asks `nova-kata` for a container.
4. **Instant Unpause:** `nova-kata` picks a paused container from the SQLite DB, SSHs into the worker, and runs `nerdctl unpause <container_name>` . Because the Kata VM is already booted and resident in RAM, resuming it takes only **~50-100ms**.
5. **Proxy Forwarding:** The gateway proxies the HTTP payload to the Worker VM's public IP on the allocated Host Port (e.g., `http://<worker_ip>:9005`).
6. **Nginx Handoff:** The Nginx instance on the Worker VM receives the traffic on `9005` and forwards it to the unpaused Kata container's internal IP.
7. **Function Execution:** The user's code inside the container runs, generating an HTTP response.
8. **Response & Return to Pool:** The response flows back through the gateway to the user. After an idle period, the gateway sends a `/release` signal to `nova-kata` . `nova-kata` runs `nerdctl pause` on the worker, freezing the VM state again so it can be reused for the next request.